DS605 · Machine Learning

Decision Trees

Lecture notes for Lectures 27 and 28

Arpit Rana · Running example: ShopKart payment methods and delivery times

How to read these notes

Lecture 27 explains how a decision tree makes predictions and how the CART algorithm grows one. Lecture 28 explains how to prevent a tree from overfitting, how trees predict numeric values, and where trees are weak.

All numbers come from scikit-learn 1.9 on the ShopKart data used in class. Each section ends with a short question to check your understanding.

Lecture 27: Growing a Decision Tree

The ShopKart payment problem

At checkout, ShopKart can pre-select the payment method that a customer is most likely to use. Cash on delivery (COD) is costly for the company: cash must be handled, and more parcels are refused at the door. If we can predict COD early, we can offer a small prepaid discount to exactly those customers.

We have 150 past orders, 50 for each payment method. Each order has two features:

- x_1 = **tenure**: how long the customer has been with ShopKart, in months;
- x_2 = **order value**: the value of the basket, in thousands of rupees.

The target is the payment method: COD, UPI, or Card. This is a **three-class** classification problem.

Order	Tenure x_1 (months)	Order value x_2 (₹ thousand)	Paid by
O-003	1.9	2.7	COD
O-008	1.0	1.8	COD
O-064	15.1	3.4	UPI
O-091	12.3	2.0	UPI

Order	Tenure x_1 (months)	Order value x_2 (₹ thousand)	Paid by
O-111	11.2	4.5	Card
O-121	31.0	6.5	Card

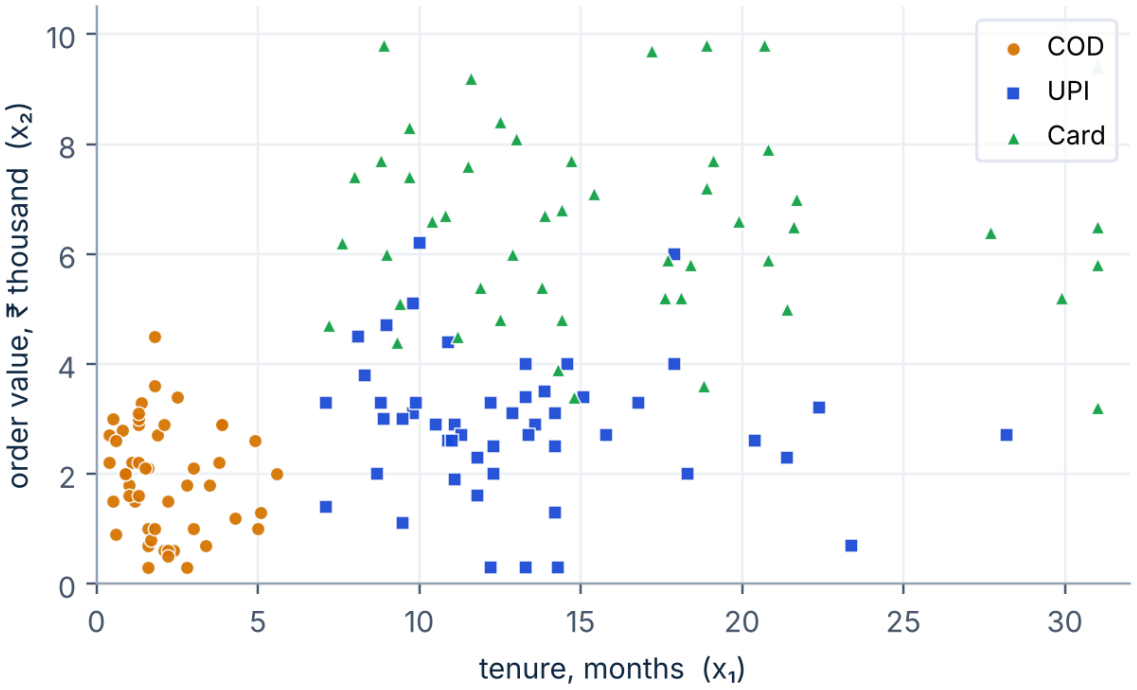


Figure 1. The 150 ShopKart orders. Amber circles are paid by COD, blue squares by UPI, and green triangles by Card.

A checkout manager who may only ask yes/no questions would probably ask two of them. First: "Has the customer been with us for less than about six months?" If yes, the answer is almost always COD, because new customers do not yet trust prepaying. Then, for regular customers: "Is the basket worth more than about ₹4,750?" Small baskets are paid by UPI and large ones by Card, often as an EMI. Two questions give three answers. That is a decision tree.

What is a decision tree?

Definition

A **decision tree** is a representation of a function that maps a vector of attribute values to a single output value, a "decision". It reaches its decision by performing a **sequence of tests**, starting at the root and following the appropriate branch until it reaches a leaf.

- Decision trees can perform both **classification** and **regression**.
- They handle binary and **multiclass** classification directly, without one-versus-rest.

- They can fit complex datasets using boundaries **parallel to the axes**.
- They are often called **interpretable** because we can read the questions.
- They need **no feature scaling**.
- They are the building blocks of **Random Forests**.

The parts of a tree

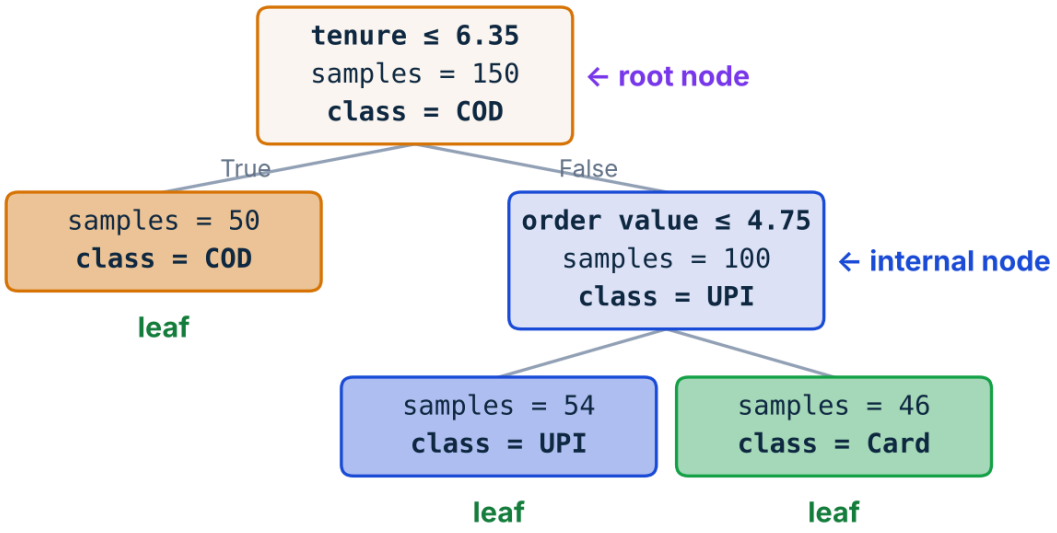


Figure 2. The checkout manager's tree with its parts labeled.

Part	Meaning
Root node	The first test. Every input starts here.
Internal node	A test of the value of **one** input feature.
Branch	One possible outcome of the test: yes (True) or no (False).
Leaf node	No test. It specifies the value to be returned by the function.

Training a tree in scikit-learn

```
from sklearn.tree import DecisionTreeClassifier

# X: tenure (months), order value (₹ thousand)
# y: 0 = COD, 1 = UPI, 2 = Card
tree_clf = DecisionTreeClassifier(max_depth=2, random_state=42)
tree_clf.fit(X, y)
```

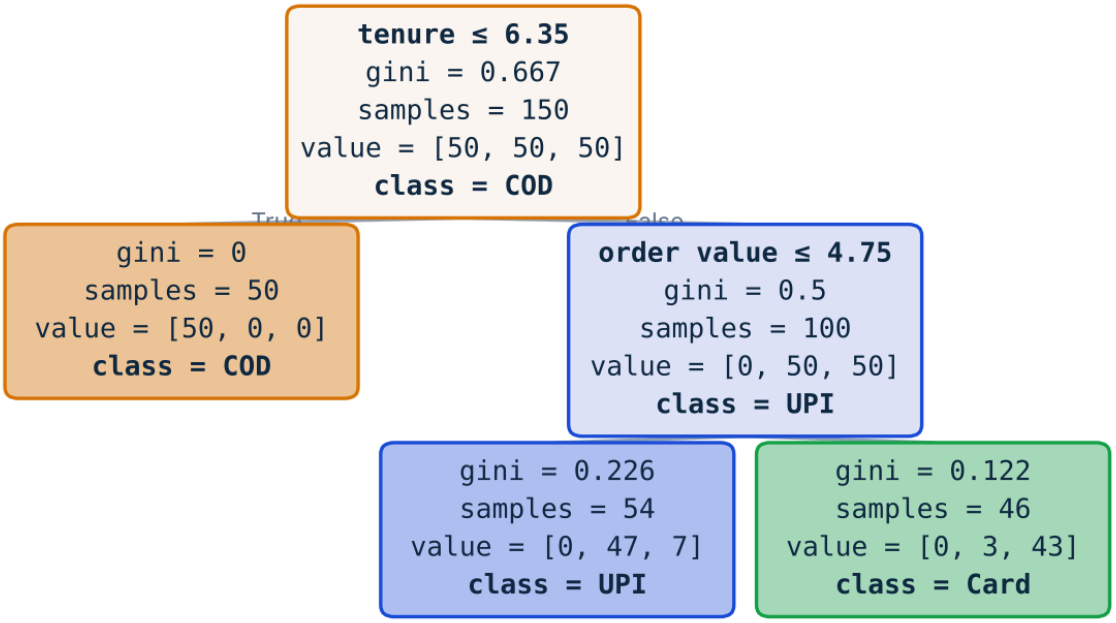


Figure 3. The fitted tree with `max_depth=2`. In each node, go left when the test is true and right when it is false.

The tree learned exactly the manager's two questions, and it chose the thresholds itself: `tenure ≤ 6.35` and `order value ≤ 4.75`. The color of a node shows its majority class; a darker color means a purer node.

Making predictions

Take a new order from a customer with 8 months of tenure and a basket of ₹4,200.

1. At the root: is tenure ≤ 6.35? **No**, so move to the right child.
2. At the internal node: is order value ≤ 4.75? **Yes**, so move to the left child.
3. This is a leaf. It holds 54 training orders, of which 47 were paid by UPI. The tree predicts **UPI**.

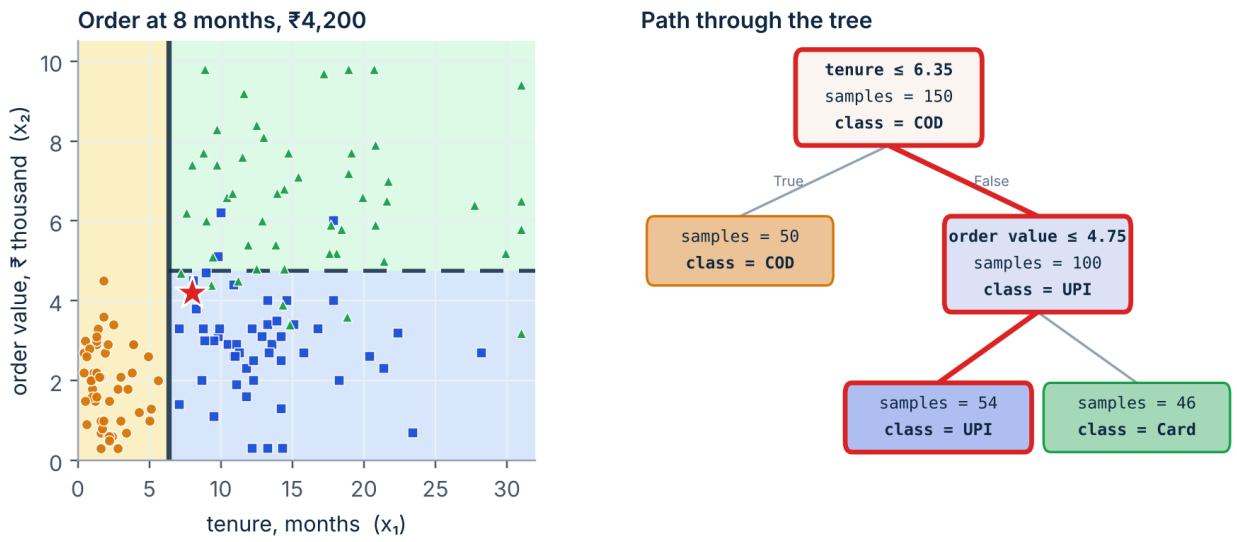


Figure 4. Left: the order (red star) in the regions of the tree. Right: its path from the root to a leaf.

Decision boundaries

Every test compares **one feature** with a threshold. So every boundary is a straight line parallel to an axis, and the tree divides the plane into rectangles. Each rectangle is one leaf.

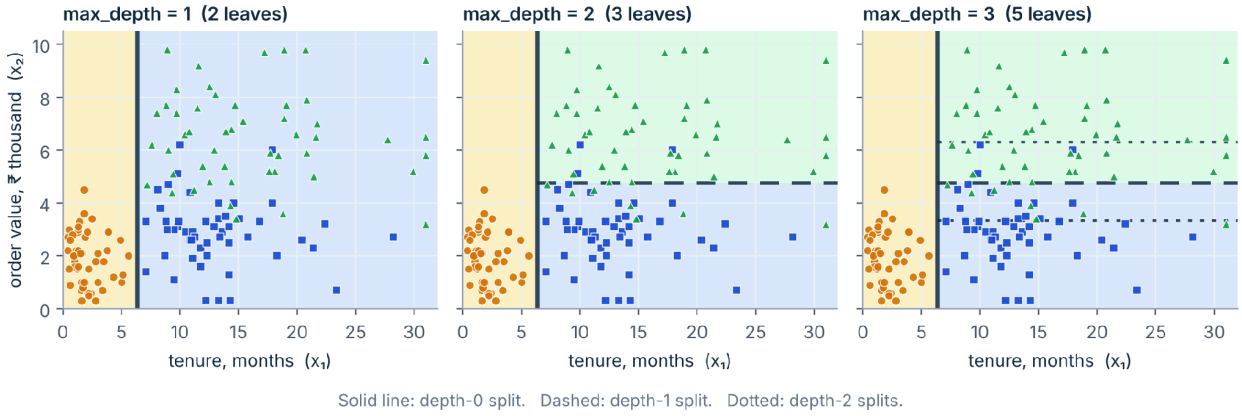


Figure 5. Decision regions for trees of depth 1, 2, and 3. The depth-0 split is at tenure = 6.35. The left region is pure COD, so it is never split again.

A customer the tree has never seen

A brand-new customer (tenure 1.5 months) places a ₹9,000 order. No COD order in our data was above ₹4,500. The tree still predicts **COD**: its first question asks only about tenure, and the basket value is never checked. The tree extends its last region to the edge of the map and does not know that it is guessing. Be careful with predictions far from the training data.

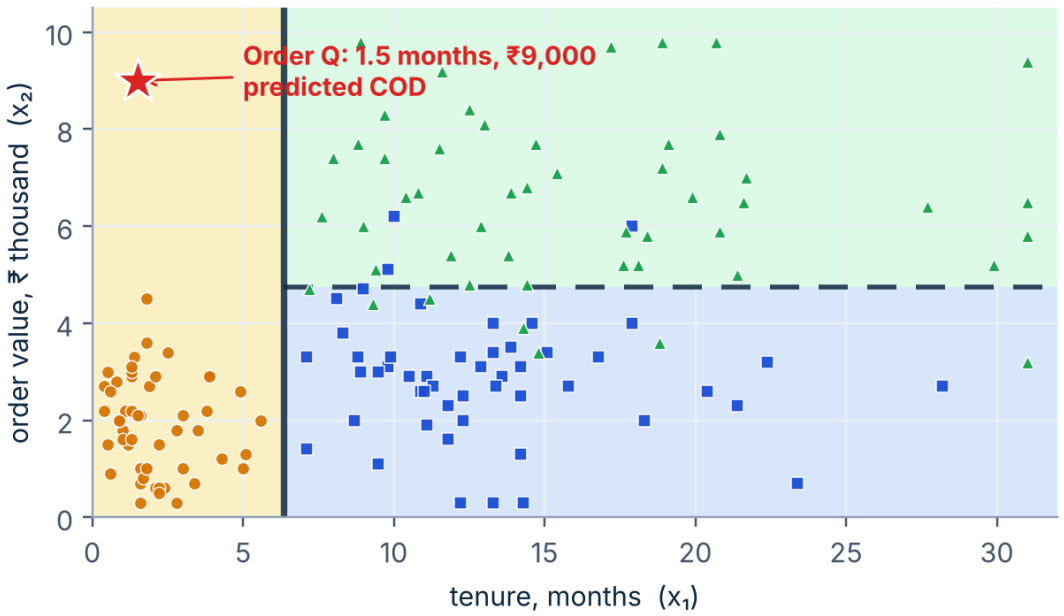


Figure 6. Order Q lies in a region with no training orders near it.

Reading the numbers in a node

Attribute	Meaning	Example (right child of the root)
`samples`	How many training orders reach this node.	100 orders with tenure > 6.35
`value`	How many of those orders are in each class, in the order [COD, UPI, Card].	[0, 50, 50]
`gini`	The node's Gini impurity is between 0 and 1. A value of 0 means every order has the same class.	0.5
`class`	The class the node would predict: the most common class in `value`.	UPI

This node holds 50 UPI and 50 Card orders, a tie. Scikit-learn breaks ties by choosing the first class in the class list, so it selects UPI. The label does not matter here because this node is not a leaf.

Estimating class probabilities

A tree can also estimate the likelihood of each class. It finds the leaf for the input and returns the fraction of training instances of each class in that leaf:

$$\hat{p_k} = \text{(number of training instances of class } k \text{ in the leaf) / (number of training instances in the leaf)}$$

For the order at 8 months and ₹4,200, the leaf holds [0, 47, 7], so

$$\hat{p}_{\text{COD}} = 0/54 = 0, \quad \hat{p}_{\text{UPI}} = 47/54 \approx 0.870, \quad \hat{p}_{\text{Card}} = 7/54 \approx 0.130$$

Every input that lands in the same leaf gets the same probabilities, even if it sits right next to a boundary.

```
tree_clf.predict_proba([[8, 4.2]]).round(3)   # array([[0.   , 0.87 , 0.13 ]])
tree_clf.predict([[8, 4.2]])                   # array([1])  →  UPI
```

Question

A node holds [0, 3, 43]. What class does it predict, and with what probability?

Answer. Card, with probability 43/46 ≈ 0.935.

Measuring impurity

A good question sends the orders into child nodes that are **pure**, that is, mostly of one class. To choose between questions, we need a number that says how mixed a node is. The two common choices are Gini impurity and entropy.

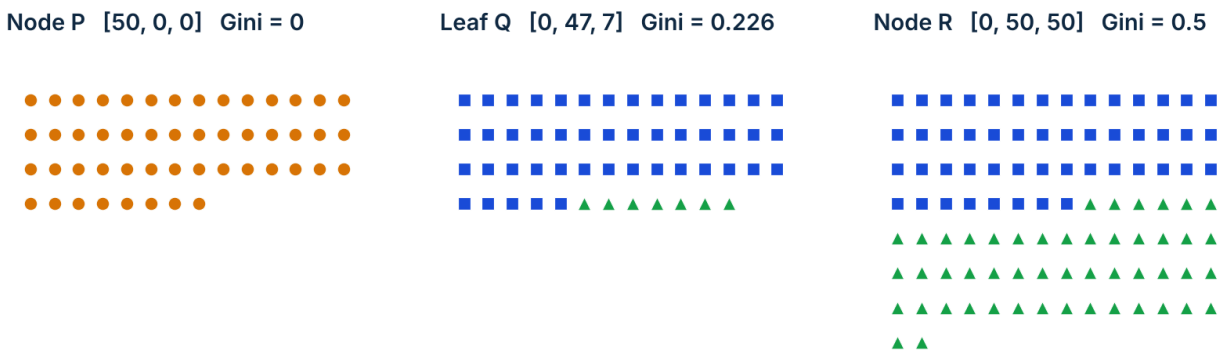


Figure 7. Three nodes from our trees. P is pure, Q is mostly UPI, and R is an even mix of two classes.

Gini impurity

The Gini impurity of the set X of training instances at a node is

$$\text{Gini}(X) = 1 - \Sigma_{i=1}^{n} p_i^2$$

Here n is the number of classes, and p_i is the probability that an instance in X belongs to class C_i. We estimate it as $p_i = |C_{i,X}| \,/\, |X|$, the share of instances at the node that are in class C_i.

Why this formula? A guessing game

Gini impurity is the probability that a random guess is wrong. Take leaf Q, with 47 UPI and 7 Card orders, so $p_{\text{UPI}} = 47/54 \approx 0.87$ and $p_{\text{Card}} = 7/54 \approx 0.13$. Play a game with two random steps:

1. **Pick one order** from the node at random. Its true class is UPI with probability 0.87 and Card with probability 0.13.
2. **Guess its label at random**, using the same mix. Imagine a bag with 47 "UPI" slips and 7 "Card" slips, and draw one slip without looking at the order.

The guess is right only when the slip matches the true class. The two steps are independent, so we multiply their probabilities:

True class	Guess	Probability
UPI	UPI (right)	$0.87 \times 0.87 = 0.757$
Card	Card (right)	$0.13 \times 0.13 = 0.017$
UPI	Card (wrong)	$0.87 \times 0.13 = 0.113$
Card	UPI (wrong)	$0.13 \times 0.87 = 0.113$

The probability of a right guess is $0.757 + 0.017 = 0.774$, which is $p_{\text{UPI}}{}^2 + p_{\text{Card}}{}^2 = \Sigma_i \, p_i{}^2$. The probability of a wrong guess is therefore

$$1 - \Sigma_i \, p_i{}^2 = 1 - 0.774 = 0.226 = \text{Gini}(Q)$$

An equivalent way to say this: pick **two** instances at random (with replacement). Gini impurity is the probability that two randomly chosen items belong to **different** classes.

- A **pure** node, such as [50, 0, 0], always gives a right guess: $\text{Gini} = 1 - 1^2 = 0$.
- An **even mix** of two classes, such as [0, 50, 50], gives a right guess only half the time: $\text{Gini} = 1 - (0.5^2 + 0.5^2) = 0.5$.
- For an even mix of n classes, the largest value is $1 - 1/n$; for three classes, it is 0.667.

Worked examples from our tree

Node	value	Calculation	Gini
Root	[50, 50, 50]	$1 - 3 \times (50/150)^2 = 1 - 3 \times 0.111$	0.667
Left leaf	[50, 0, 0]	$1 - (50/50)^2$	0
Leaf Q	[0, 47, 7]	$1 - (47/54)^2 - (7/54)^2 = 1 - 0.758 - 0.017$	0.226
Right leaf	[0, 3, 43]	$1 - (3/46)^2 - (43/46)^2$	0.122

Every question the tree asked made the nodes purer: 0.667 at the root, down to 0, 0.226, and 0.122 at the leaves.

Entropy

Entropy is the second common measure. In Lecture 11–12, we met it as a measure of how uncertain an outcome is. The entropy of a node is

$$H(X) = -\Sigma_{i:\, pi>0}\, p_i \log_2 p_i$$

It is 0 for a pure node and $\log_2 n$ for an even mix of n classes. For our root and for leaf Q:

$$H(\text{root}) = -3 \times (1/3) \log_2 (1/3) = \log_2 3 \approx 1.585$$

$$H(Q) = -(47/54) \log_2 (47/54) - (7/54) \log_2 (7/54) \approx 0.174 + 0.382 = 0.556$$

The drop in entropy from a parent to its size-weighted children is called the **information gain**. The ID3 algorithm chooses the question with the largest gain. In scikit-learn, write `DecisionTreeClassifier(criterion="entropy")`.

Gini or entropy? Different numbers, same choice

For a node with two classes, where p is the share of class 1, the two measures and the error rate are:

$$\text{Gini} = 2p(1-p), \qquad H = -p \log_2 p - (1-p) \log_2 (1-p), \qquad \text{error} = \min(p, 1-p)$$

Share p	Gini	Entropy	Error rate
0 (pure)	0	0	0
0.1	0.18	0.47	0.10
0.3	0.42	0.88	0.30
0.5 (even mix)	0.50	1.00	0.50

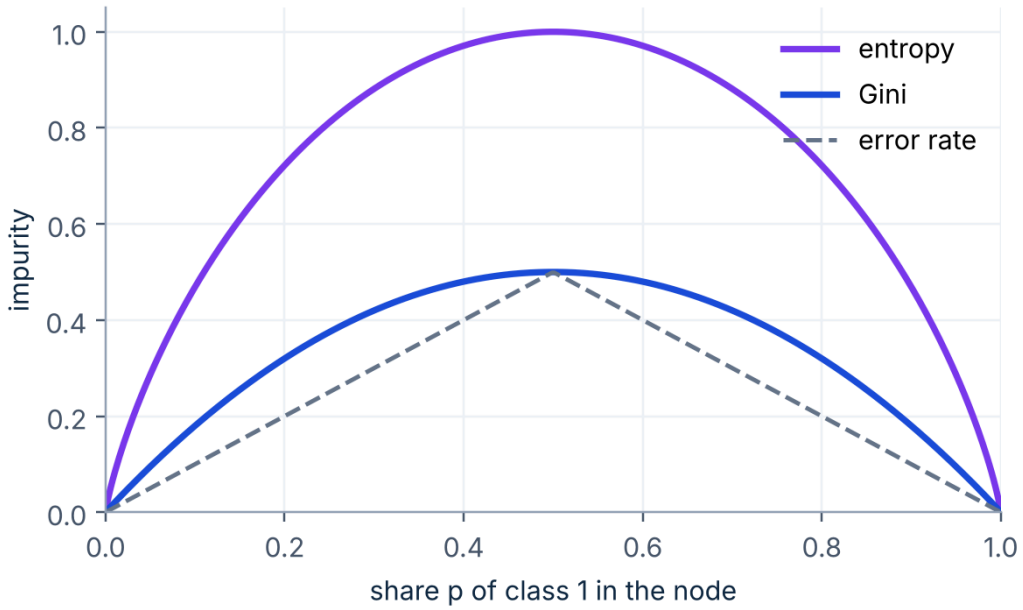


Figure 8. Gini impurity, entropy, and the error rate for a node with two classes.

- **Both measure how mixed a node is.** Both are 0 for a pure node and largest at a 50/50 mix. Entropy is about twice as tall, but the shape is the same.
- **The scale does not matter.** CART never uses an impurity value on its own; it only compares candidate splits. Gini and entropy almost always rank the splits in the same order. On our 150 orders, both criteria grow exactly the same depth-2 tree.
- **In practice**, Gini is the default because it is slightly faster (since it does not involve logarithms). Entropy sometimes gives slightly more balanced trees. The choice rarely changes the result.

Why not use the error rate? The error rate is a straight line, so it often cannot tell a better split from a worse one. Suppose a node has 400 UPI and 400 Card orders:

	Split A: (300, 100) and (100, 300)	Split B: (200, 400) and (200, 0)
Weighted error rate	0.25	0.25 (a tie)
Weighted Gini	0.375	0.333 (B is better)

Split B creates a completely pure node, which is the more useful split. Gini and entropy reward this because their curves bend; the error rate does not see the difference.

Question

A node holds [25, 25]. Give its Gini impurity and its entropy.

Answer. Gini = 1 − 0.25 − 0.25 = 0.5. Entropy = 1 bit. Both are at their largest for two classes.

How CART grows a tree

Scikit-learn trains decision trees using the **CART** (Classification and Regression Tree) algorithm. It first splits the training set into two subsets, using a single feature k and a threshold t_k. To choose them, it tries every pair (k, t_k) and keeps the pair that produces the purest subsets, weighted by their size.

The cost function

A candidate split sends the m instances at the node into X_{left} (instances with $x_k \le t_k$) and X_{right} (instances with $x_k > t_k$). CART minimises

$$J(k, t_k) = (m_{\text{left}} / m) \cdot \text{Gini}(X_{\text{left}}) + (m_{\text{right}} / m) \cdot \text{Gini}(X_{\text{right}})$$

Here, m_{left} and m_{right} are the numbers of instances in the two subsets. The cost is a **weighted average**: each side counts in proportion to its size, so a tiny pure side cannot hide a large mixed one. Candidate thresholds lie halfway between neighboring feature values.

A worked example with ten orders

Take ten orders, A to J: three COD, four UPI and three Card. The parent node has

$$\text{Gini(parent)} = 1 - 0.3^2 - 0.4^2 - 0.3^2 = 0.66$$

Compare the first questions.

	tenure ≤ 5.75	**order value ≤ 4.8**
Left side	{A, B, C}: 3 COD, Gini = 0	3 COD + 3 UPI, Gini = $1 - 0.5^2 - 0.5^2 = 0.500$
Right side	4 UPI + 3 Card, Gini = $1 - (4/7)^2 - (3/7)^2 = 0.490$	1 UPI + 3 Card, Gini = $1 - (1/4)^2 - (3/4)^2 = 0.375$
Cost J	$(3/10)(0) + (7/10)(0.490) = 0.343$	$(6/10)(0.500) + (4/10)(0.375) = 0.450$

The tenure split has a lower cost. CART tries all 18 candidate thresholds (9 for each feature), and no split does better, so `tenure ≤ 5.75` becomes the root question.

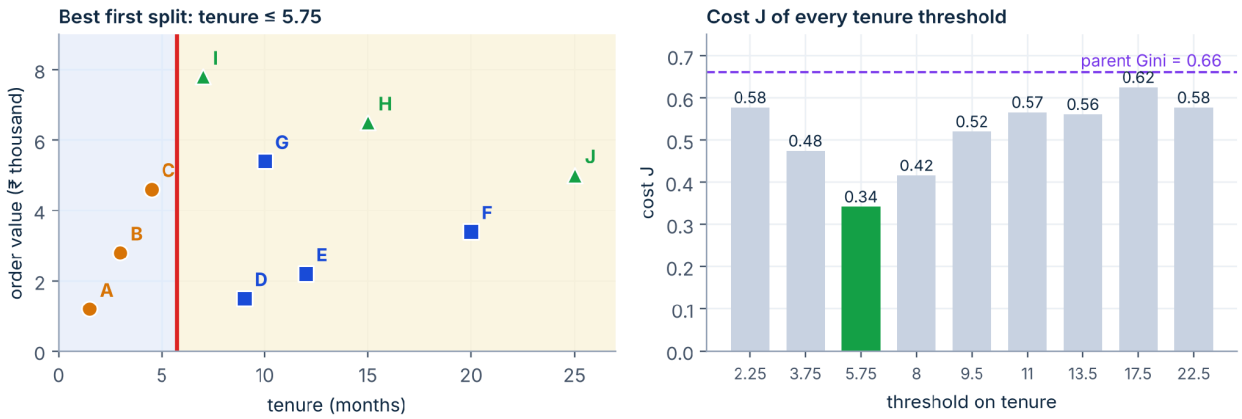


Figure 9. Left: the ten orders and the best split. Right: the cost J of every tenure threshold; the green bar is the minimum.

The CART algorithm

1. **Try every split.** For each feature x_k and each threshold t_k, split X into X_{left} and X_{right} and compute $J(k, t_k)$.
2. **Keep the best one**: the feature and threshold with the lowest cost.
3. **Stop?** If a stopping rule holds (the node is pure, the maximum depth is reached, or no split reduces the impurity), make the node a leaf and return.
4. **Otherwise, recurse.** Apply CART to X_{left}, then to X_{right}.

Property	Meaning
Binary	Every question has exactly two answers: yes or no.
Recursive	The same procedure is repeated for each child until a stopping rule is satisfied.
Greedy	It takes the best split **now**. It does not check whether this split leads to the lowest impurity a few levels down.

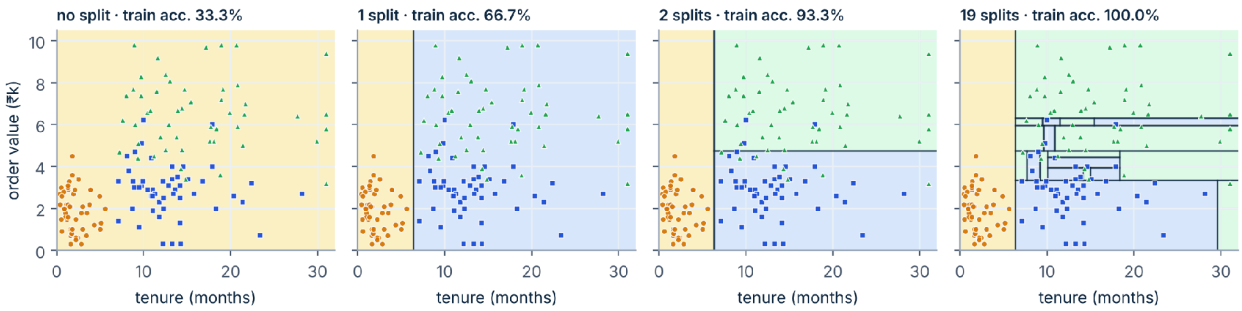


Figure 10. CART is growing a tree on the 150 orders. With no limit, it keeps splitting until every leaf is pure (19 splits, 20 leaves).

A greedy algorithm can choose a bad first question

ShopKart ran a "Buy more, save more" offer. New customers with small baskets responded, as did loyal customers with large baskets; the other two groups did not. Any single split leaves both sides about half-and-half.

- Greedy CART picks the split with the tiniest gain, `tenure ≤ 1.45` ($J = 0.448$), which cuts off only three customers. With two question levels, the tree is only **62.5%** accurate.
- The useful first split, `tenure ≤ 11.5`, yields **no gain** ($J = 0.500$). Yet after one more split on order value, it gives **100%** accuracy. A greedy algorithm cannot see this.

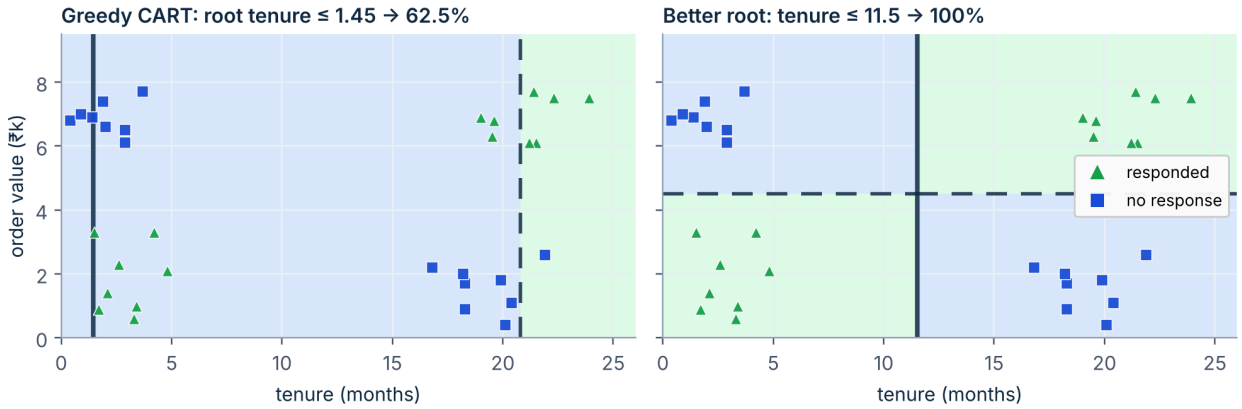


Figure 11. The offer data with two depth-2 trees. Green triangles responded; blue squares did not.

Why not search for the best tree?

Finding the optimal tree is an **NP-complete** problem: it requires $O(\exp(m))$ time, making it intractable even for small training sets. CART gives up the guarantee of the best tree; in return, it runs fast and usually finds a reasonably good one.

Algorithm	Splits	Impurity measure	Notes
ID3	Many branches per node	Entropy (information gain)	Categorical features only
C4.5	Many branches per node	Entropy (gain ratio)	Adds numeric thresholds, missing values and pruning
CART (scikit-learn)	Binary only	Gini or entropy; MSE for regression	One algorithm for classification and regression

Inspecting a tree in Python

```
from sklearn.tree import export_text, plot_tree

print(export_text(tree_clf, feature_names=["tenure", "order_value"]))
# |--- tenure <= 6.35
# |   |--- class: 0
# |--- tenure >  6.35
# |   |--- order_value <= 4.75
# |   |   |--- class: 1
# |   |--- order_value >  4.75
# |   |   |--- class: 2

plot_tree(tree_clf, filled=True)     # draws the diagram with matplotlib
```

Summary of Lecture 27

- A tree predicts by asking **yes/no questions** from the root down to a leaf.
- Each question tests **one feature**, so every boundary is parallel to an axis.
- Gini impurity $1 - \Sigma_i \, p_i^2$ and entropy both score how mixed a node is.
- CART picks the split with the lowest **size-weighted** impurity, then repeats on each side.
- CART is **greedy**: fast and usually good, but not guaranteed to find the best tree.

Question

A split sends 2 orders to a pure left node and 98 mixed orders to the right. Its left Gini is 0. Is it a good split?

Answer. Usually not. The cost J weights each side by its size, so the right side counts for 98%. The split barely lowers the impurity.

Lecture 28: Taming a Decision Tree

Overfitting

Decision trees make very few assumptions about the training data. Left unconstrained, CART keeps splitting until every leaf is pure. On our 150 orders, this gives 20 leaves and a depth of 11. It classifies every training order correctly, yet on 600 new orders, it is only **88.7%** accurate. The depth-2 tree with only 3 leaves achieves **93.8%**.

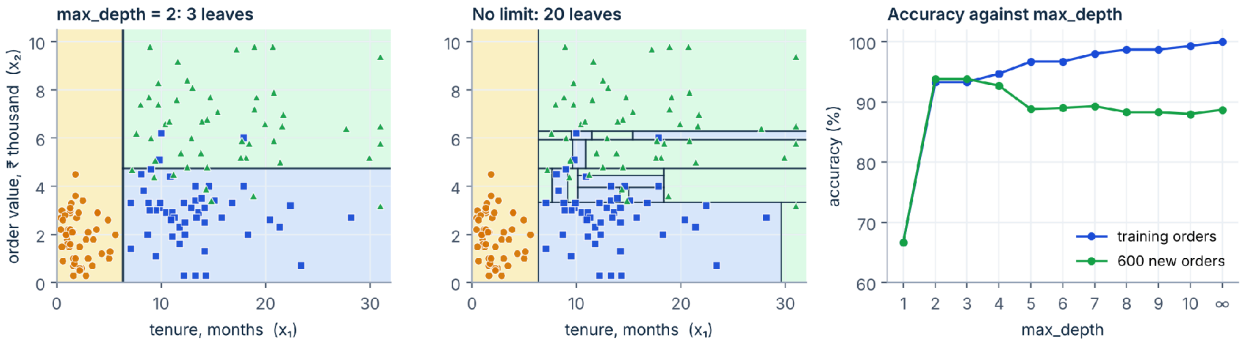


Figure 12. Left and center: a depth-2 tree and an unrestricted tree. Right: accuracy on training orders keeps rising with depth, while accuracy on new orders falls after depth 3.

The small rectangles in the unrestricted tree chase a few unusual orders, such as a Card payment for a ₹3,200 basket. They are noise, not patterns. The tree has **overfitted**. `max_depth` is a hyperparameter: a larger value makes the model more complex and can lead to overfitting.

max_depth	1	2	3	4	5	6	8	None
Leaves	2	3	5	8	11	14	15	20
Training accuracy	66.7%	93.3%	93.3%	94.7%	96.7%	96.7%	98.7%	100%
New orders	66.7%	93.8%	93.8%	92.7%	88.8%	89.0%	88.3%	88.7%

Regularisation hyperparameters

To avoid overfitting, we restrict the tree's freedom during training. This is **regularisation**. A tree has no weights to shrink (as Ridge and Lasso did in Lecture 16–17), so we limit its **shape** instead.

Hyperparameter	What it limits
`max_depth`	The maximum depth of the tree.

Hyperparameter	What it limits
`min_samples_split`	The minimum number of samples a node must have before it can be split.
`min_samples_leaf`	The minimum number of samples every leaf must have.
`max_leaf_nodes`	The maximum number of leaves.
`max_features`	The maximum number of features evaluated for splitting at each node.

Rule of thumb

Increasing the `min_*` hyperparameters or reducing the `max_*` hyperparameters regularises the model.

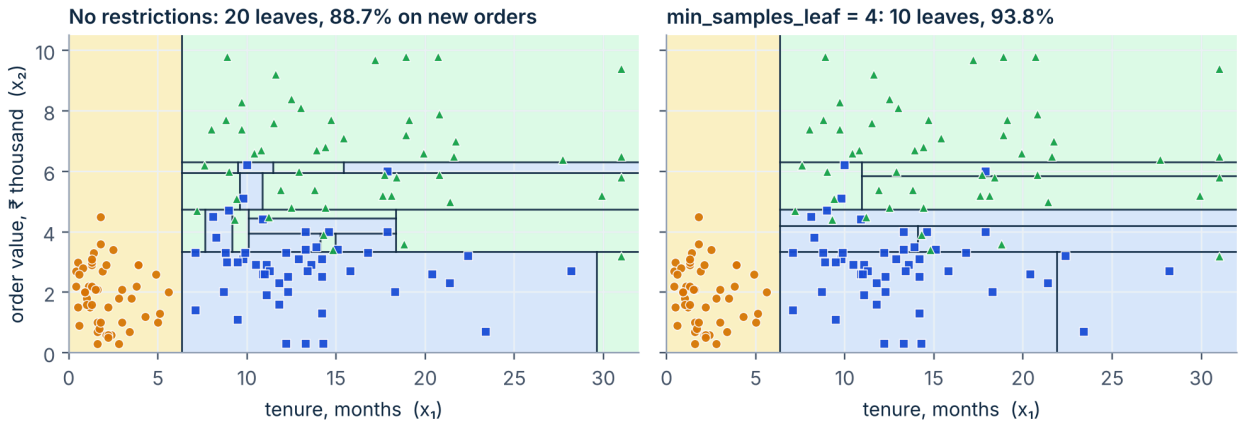


Figure 13. Left: no restrictions (clearly overfitting). Right: every leaf must hold at least 4 orders; the small islands disappear, and the model generalizes better.

Post-pruning

The hyperparameters above stop the tree **from growing** (pre-pruning). The alternative is to grow the full tree and then cut back the branches that do not pay for themselves (post-pruning). Scikit-learn uses **cost-complexity pruning**, which charges a price α for every leaf:

$$R_\alpha(T) = R(T) + \alpha \cdot |\text{leaves}(T)|$$

Here, $R(T)$ is the total weighted impurity of the leaves of tree T. A larger α removes more branches. The hyperparameter is `ccp_alpha`.

ccp_alpha	0	0.0044	0.0057	0.0063	0.013	0.0183	0.2147
Leaves	20	18	13	5	4	3	2
Training accuracy	100%	99.3%	98.0%	94.0%	93.3%	93.3%	66.7%
New orders	88.7%	88.2%	90.7%	93.2%	93.8%	93.8%	66.7%

Choosing the hyperparameters

```
from sklearn.model_selection import GridSearchCV

params = {"max_depth": [2, 3, 4, 5, 6, None],
          "min_samples_leaf": [1, 2, 4, 8, 16]}
search = GridSearchCV(DecisionTreeClassifier(random_state=42), params, cv=5)
search.fit(X, y)

search.best_params_         # {'max_depth': 2, 'min_samples_leaf': 1}
search.best_score_          # 0.913  (5-fold cross-validation)
search.score(X_new, y_new)  # 0.938  (600 new orders)
```

Never choose `max_depth` by training accuracy: it always prefers the deepest tree. Use validation folds. `ccp_alpha` can be tuned in the same way.

> **Question**
>
> A tree scores 100% on training data and 80% on validation data. Should you raise or lower `max_depth`?
>
> **Answer.** Lower it. The tree is overfitting, so it needs to be simpler. If instead both scores were 70%, the tree would be underfitting; then allow a deeper tree or a smaller `min_samples_leaf`.

Computational complexity

Task	Cost	Why
Prediction	$O(\log_2 m)$	One question per level, and a roughly balanced tree has about $\log_2 m$ levels. The cost does not depend on the number of features.
Training	$O(n \times m \log_2 m)$	At each level, CART compares every feature on every training instance (with sorting). Limits such as `max_depth` make it faster.

Here n is the number of features and m is the number of training instances. For one lakh orders, $\log_2 10^5 \approx 17$, so a balanced tree needs about 17 questions per prediction. Multiplying the data by 1,000 adds only about 10 more questions.

Regression trees

Decision trees can also predict numbers. We return to the ShopKart delivery-time data from Lecture 16–17: 200 delivered orders, with the distance from the warehouse (km) as the feature and the delivery time (hours) as the target.

```
from sklearn.tree import DecisionTreeRegressor

tree_reg = DecisionTreeRegressor(max_depth=2, random_state=42)
tree_reg.fit(X, y)          # X: distance (km), y: delivery time (hours)
```

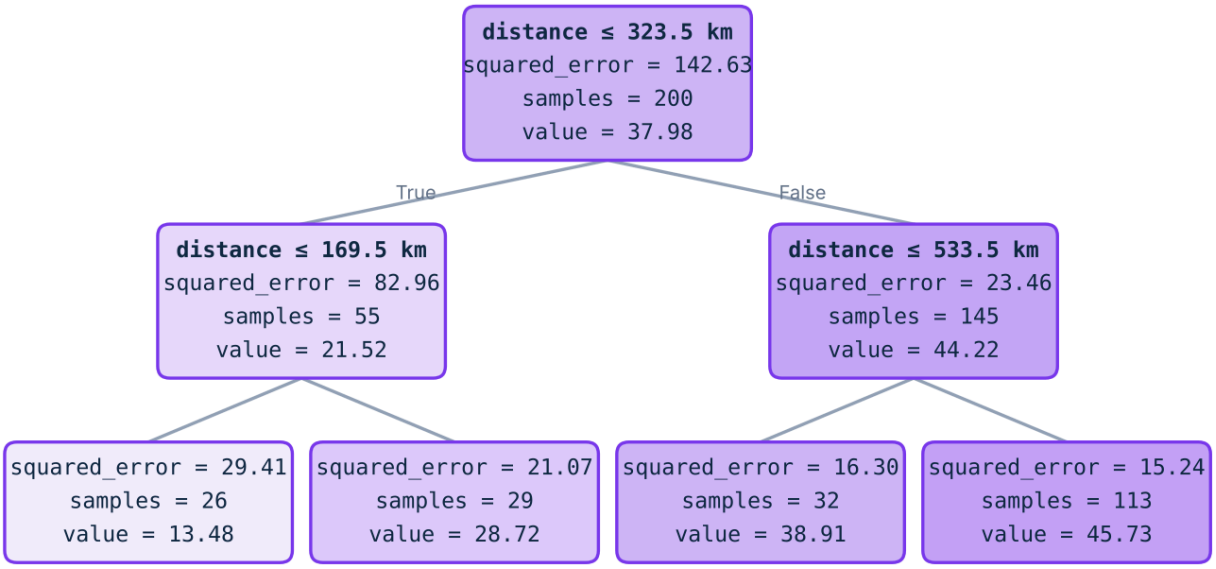


Figure 14. The fitted regression tree. Older versions of scikit-learn print `MSE` instead of `squared_error`.

The tree looks like a classification tree, but each node predicts a **value rather** than a class. The value of a node is the **mean target** of the training instances that reach it:

$$\hat{y}_{\text{node}} = (1 / m_{\text{node}}) \, \Sigma_{i \in \text{node}} \, y^{(i)}$$

For an order 400 km away: is distance ≤ 323.5? No, go right. Is distance ≤ 533.5? Yes, go left. The leaf contains 32 orders, with a mean delivery time of **38.9 hours**. That is the prediction.

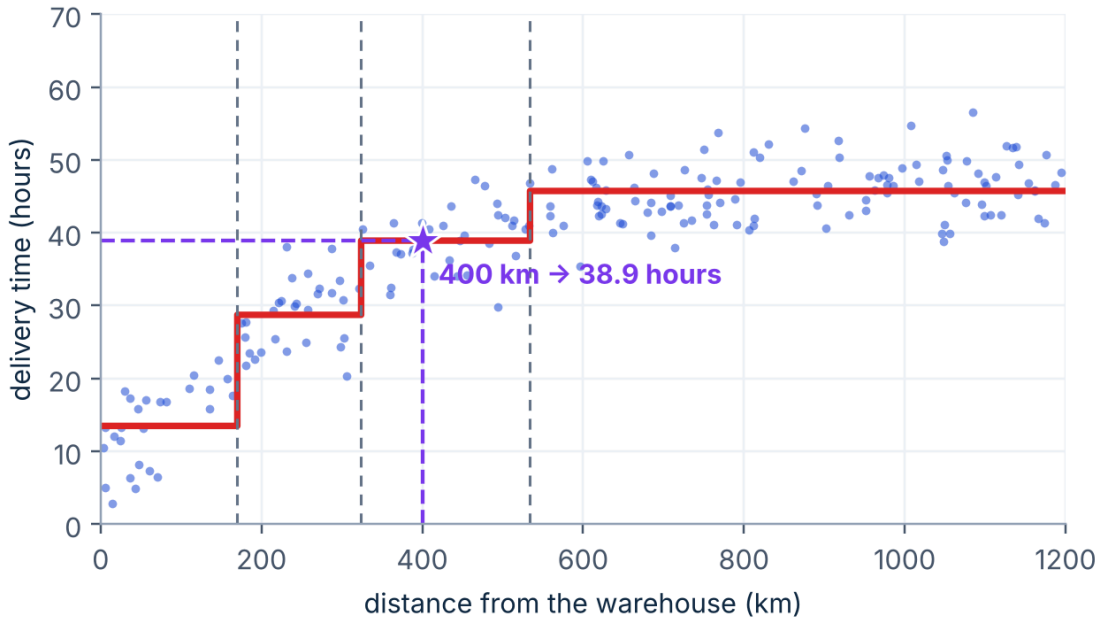


Figure 15. The depth-2 regression tree is a step function with four steps: 13.5, 28.7, 38.9, and 45.7 hours.

The cost function for regression

CART works in the same way as before, except that it splits the training set to minimize the **mean squared error** in place of the impurity:

$$J(k, t_k) = (m_{\text{left}} / m) \cdot \text{MSE}_{\text{left}} + (m_{\text{right}} / m) \cdot \text{MSE}_{\text{right}}$$

$$\text{where } \text{MSE}_{\text{node}} = (1 / m_{\text{node}}) \, \Sigma_{i \in \text{node}} (\hat{y}_{\text{node}} - y^{(i)})^2$$

At the root, the MSE about the mean (37.98 hours) is 142.6. The best split, distance ≤ 323.5 km, lowers the weighted MSE to 39.8.

Regression trees overfit too

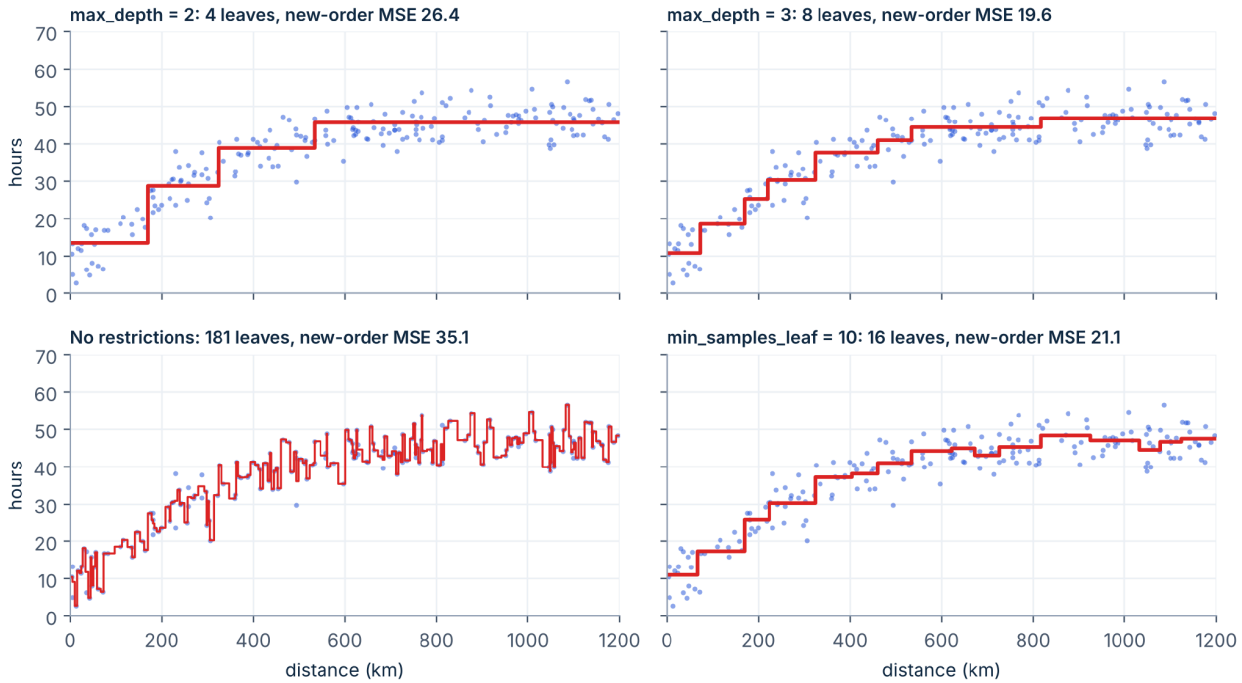


Figure 16. Four regression trees on the same orders. Without restrictions, the tree memorizes the noise; `min_samples_leaf = 10` gives a sensible staircase.

Setting	Leaves	Training MSE	MSE on 300 new orders
max_depth = 2	4	18.1	26.4
max_depth = 3	8	14.3	19.6
No restrictions	181	2.0	35.1
min_samples_leaf = 10	16	14.3	21.1
Linear regression (for comparison)	–	–	43.1

Question

A regression leaf holds delivery times of 20, 24 and 31 hours. What does it predict?

Answer. The mean: (20 + 24 + 31) / 3 = 25 hours.

Strengths of decision trees

Strength	Explanation
The model is interpretable	We can display the tree, and people can see what has been learned. ShopKart's team can check the rule "new customers pay COD" against their experience.
Each prediction is explainable	We can display the path: "predicted UPI because tenure > 6.35 months and basket ≤ ₹4,750".
No feature scaling	Each test compares one feature with a threshold. Measuring tenure in days instead of months shifts the threshold (6.35 months becomes about 193 days) but sends the same orders to both sides. An SVM, in contrast, would change.

Feature importance

The importance of a feature is the total decrease in impurity it produces, summed over all nodes that split on it. Each decrease is weighted by the number of samples at the node, and the importances are scaled so that they sum to 1. For the depth-2 tree:

$$\text{root (tenure):}\quad 150(0.667) - 50(0) - 100(0.5) = 50.0$$

$$\text{node 2 (order value):}\quad 100(0.5) - 54(0.226) - 46(0.122) = 32.2$$

$$\text{importance(tenure)} = 50.0 \,/\, (50.0 + 32.2) = 0.608$$

$$\text{importance(order value)} = 32.2 \,/\, (50.0 + 32.2) = 0.392$$

```
tree_clf.feature_importances_.round(3)    # array([0.608, 0.392])
```

Treat importances as a summary, not as proof of cause. When two features are correlated, the credit can go to either one.

Weaknesses of decision trees

Sensitivity to rotation

Trees prefer boundaries parallel to the axes. Consider the question "Is a customer's spending growing?". The true rule is a diagonal: this month's spend > last month's spend. The tree has to approximate the diagonal with a staircase of 12 leaves, and it scores only 85.5% on new customers. If we first rotate the data with **PCA** (Lecture 20–21), the diagonal becomes horizontal; one split is enough, and accuracy rises to 98.8%.

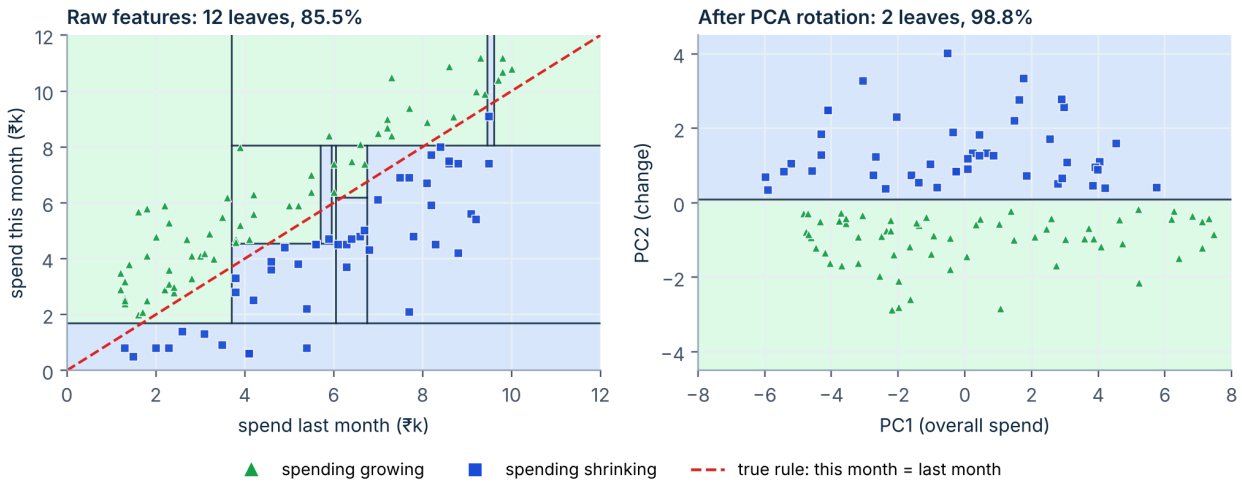


Figure 17. Left: a staircase approximates the diagonal rule. Right: after the PCA rotation, a single cut separates the classes.

Instability

Trees are very sensitive to small variations in the training data. Removing a single order (O-138: 31 months, ₹3,200, Card) changes the prediction on 4.8% of the map. Training on different random 80% samples of the same 150 orders gives visibly different trees, with accuracies from 87.7% to 92.5%.

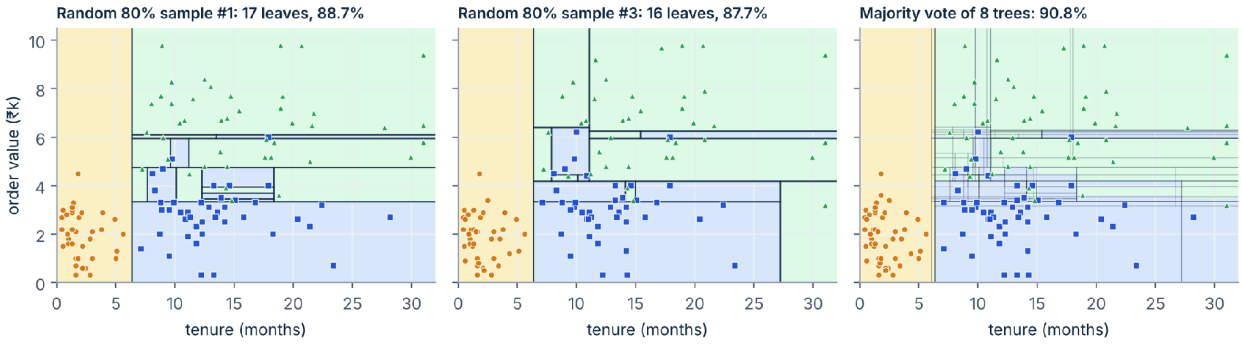


Figure 18. Two trees were trained on random 80% samples of the same orders, and the majority vote of eight such trees.

If we let eight such trees **vote**, the boundaries settle, and accuracy on new orders rises to 90.8%, above the average single tree (89.4%). Averaging many trees is the idea behind **Random Forests**, the topic of the next lecture.

Parametric and non-parametric models

	Parametric	Non-parametric
Number of parameters	Fixed before training; it does not depend on the number of training instances m.	Not known in advance; it may grow with the training set.

	Parametric	Non-parametric
Examples	Linear regression: $n + 1$ parameters. Polynomial regression of degree d: $(n + d)!\ /\ (n!\ d!)$ parameters.	Decision tree: In a sense, the nodes are the parameters. kNN: In a sense, the neighbors are the parameters.

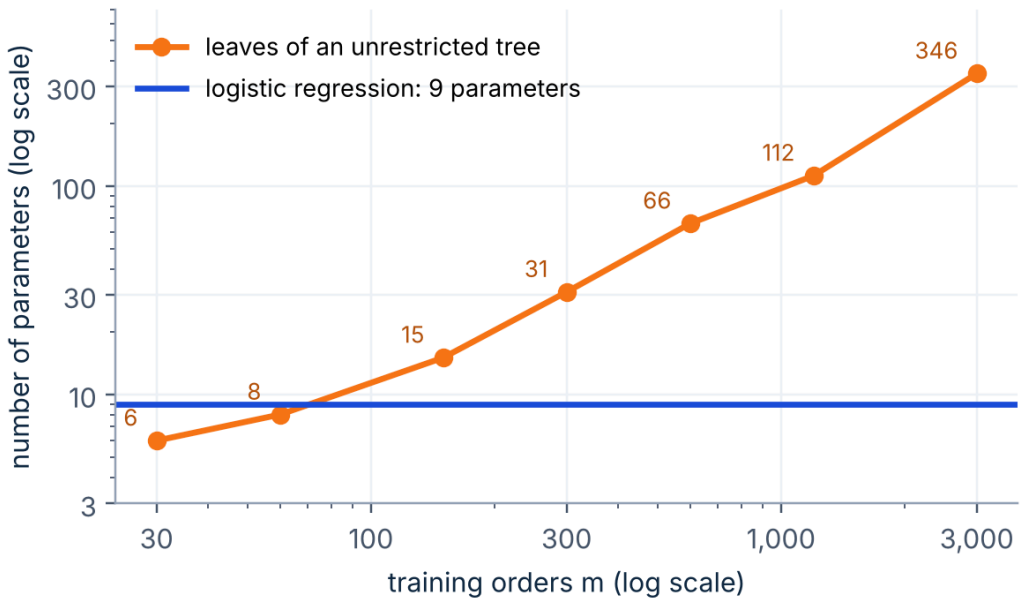


Figure 19. Unrestricted trees on more and more ShopKart orders gain roughly one leaf for every nine orders, while logistic regression always has 9 parameters.

Does this freedom mean a lower validation error? **No.** Freedom to grow is also freedom to overfit. So we constrain CART with a maximum depth, a minimum leaf size, or pruning, just as we avoid very small values of k in kNN.

Trees compared with the models we know

	Logistic regression	SVM (RBF kernel)	Decision tree
Boundary	One straight line	Smooth curves	Axis-parallel steps
Needs feature scaling	Yes, for regularisation	Yes	No
Multiclass	Softmax	One-versus-one	Directly
Can we read the model?	Through its weights	Hard	Yes, if it is small
Stability	High	High	Low

	Logistic regression	SVM (RBF kernel)	Decision tree
Main controls	C or λ	C, γ	Depth, leaf size, and pruning
ShopKart payments (600 new orders)	93.0%	90.5%	93.8% (depth 2)

Summary

1. A tree predicts by asking **one-feature yes/no questions** from the root to a leaf.
2. A leaf returns the **majority class** (and the class fractions as probabilities), or the **mean** for regression.
3. CART chooses the split with the lowest **size-weighted** Gini impurity, entropy or MSE.
4. CART is **greedy** and **binary**: fast, but not guaranteed to find the optimal tree.
5. Unrestricted trees overfit. Limit the depth or leaf size, or prune with `ccp_alpha`.
6. Trees need **no feature scaling** and are easy to explain.
7. Trees draw staircases and are **unstable**. Letting many trees vote fixes much of this.

Practice questions

Question

Why does a tree not need feature scaling, while an SVM does?

Answer. A tree compares one feature at a time with a threshold, so rescaling a feature moves the threshold but not the split. An SVM measures distances that combine the features, so their units matter.

Question

Your tree changes a lot when you retrain it on a fresh sample. Name two remedies.

Answer. Regularise it (a smaller depth, a larger `min_samples_leaf`, or pruning), or train many trees and let them vote (an ensemble such as a Random Forest).

Question

Is a node with value [0, 50, 50] purer or less pure than one with [10, 40, 50]? Use Gini impurity.

Answer. The first has Gini $1 - 0.25 - 0.25 = 0.5$. The second has $1 - 0.01 - 0.16 - 0.25 = 0.58$. The first node is purer.